SPLITSTACK

Handover prompts for finishing the project

Copy-paste prompts for an AI coding agent (Google Antigravity or similar) to complete Review 2 (JavaScript), Review 3 (Node.js + MongoDB) and the wrap-up.

Prepared by: Mahaprabhu K
Repository: https://github.com/MahaprabhuK/SplitStack
Live site: https://mahaprabhuk.github.io/SplitStack/
Work branch: refurbish (main kept identical)
Status: Review 1 done; P1-P15 remaining
Date: 2026-10-15

1. How to use this document

Give the agent one prompt at a time, in order. Each prompt is self-contained and ends with the agent logging, committing and pushing its work, then stopping. Check the result yourself before pasting the next one.

- Start every new agent conversation with Prompt 0 (project context). Paste the step prompt only after the agent has replied with its summary.
- In Antigravity, open the repository folder as the workspace and paste prompts into the Agent panel. Use the planning mode (not the fast mode) if your version offers both, and the strongest model available.
- Let the agent run terminal commands, but read what it plans before approving deletes, pushes and installs.
- After each step, open the app (npm run client, or the server from P9 on) and click through what changed.
- If the agent drifts, use the Recovery prompt in section 5. At the start of a new session, use the Status check prompt.
- Text copied from a PDF may break lines in odd places. That is fine for an agent; the meaning is unchanged.

2. One-time setup (do this yourself)

- Install Node.js 20 LTS or newer, Git, Python 3 (then: pip install reportlab, used to build the PDF log) and Antigravity.
- Ask Mahaprabhu to add your GitHub account as a collaborator on MahaprabhuK/SplitStack (Settings > Collaborators) so you can push. Accept the email invite.
- For Review 3 you need a free MongoDB Atlas cluster: create a database user, allow your IP under Network Access, and copy the connection string with a database name ending in -dev (e.g. splitstack-dev). You will paste it into server/.env yourself in step P9; do not paste it into the chat.
- Pushing a GitHub Actions workflow (step P14) needs a token with the "workflow" scope: gh auth login --scopes workflow (or gh auth refresh -s workflow).

```
git clone https://github.com/MahaprabhuK/SplitStack.git
cd SplitStack
git checkout refurbish
npm install
npm run lint:html # should report no errors
npm run client # open http://localhost:5500
```

If the site opens and the HTML check passes, you are ready to start.

Roadmap

Step	Stage	Title	Result
P0	All	Project context (paste first in every session)	Agent summary
P1	Review 2	JavaScript architecture and app shell	Commit + log
P2	Review 2	Client data layer backed by localStorage	Commit + log
P3	Review 2	Render pages from data	Commit + log
P4	Review 2	Forms, validation and CRUD flows	Commit + log
P5	Review 2	Group splitting and the debt-simplification algorithm	Commit + log
P6	Review 2	Charts, budgets, forecast, insights, search and notifications	Commit + log
P7	Review 2	Shopping checklist and receipt scan preview	Commit + log

Step	Stage	Title	Result
P8	Review 2	Review 2 QA, log and commit	Commit + log
P9	Review 3	Express server and MongoDB connection	Commit + log
P10	Review 3	Mongoose data models	Commit + log
P11	Review 3	Authentication and REST API	Commit + log
P12	Review 3	Receipt upload and server-side OCR	Commit + log
P13	Review 3	Connect the client to the API	Commit + log
P14	Review 3	Seed data and automated tests	Commit + log
P15	Wrap-up	Remove legacy code, document and deploy	Commit + log

3. Prompt 0 - Project context

Paste this first, in every new agent conversation. It contains the rules every step relies on: logging, committing, pushing and redeploying the site.

```
You are taking over SplitStack, a Web Technology course project. Read this whole message and the files it
names, then reply with a short summary of what you understood and a list of anything that looks wrong or
missing. Do not write or change any code in this turn. Wait for my next prompt.

== WHAT SPLITSTACK IS ==
An expense manager in Indian rupees (INR): personal expenses and income, categories, monthly budgets,
recurring payments, group expenses with equal/custom splits and a settle-up screen that minimises the
number of payments (debt simplification), shopping lists that convert into expenses, receipt scanning
with OCR, analytics charts, a spending forecast, smart insights, global search and notifications.

== COURSE REQUIREMENTS (each review must be demonstrable on its own) ==
- Review 1: HTML + CSS ........................... DONE
- Review 2: JavaScript (vanilla ES modules) ....... NOT STARTED  (steps P1-P8)
- Review 3: Node.js + Express + MongoDB ........... NOT STARTED  (steps P9-P14)
- Wrap-up: clean-up, docs, deploy ................. NOT STARTED  (step P15)
The original app was Python/Flask + SQLite. It is being rebuilt. The Flask files are still in the
working tree as a reference for features and business rules; they are also preserved on the `legacy`
branch and will be deleted from main in P15.

== REPOSITORY ==
- GitHub: https://github.com/MahaprabhuK/SplitStack
- Work branch: `refurbish`. `main` is kept identical to it (push both). `legacy` = old Flask app:
  never commit to it, never rewrite it. Never force-push any branch.
- Live static site: https://mahaprabhuk.github.io/SplitStack/
  It is served by GitHub Pages from the `gh-pages` branch, which contains ONLY the client/ folder
  (made with `git subtree split --prefix client`). There is no GitHub Actions deploy workflow.

== READ THESE FILES FIRST ==
docs/REFURBISH_PLAN.md   plan, decisions, what each review contains
client/README.md         Review 1 structure and the CSS techniques used
docs/prompt_log.json     the prompt log: "entries" = done, "planned" = the remaining steps P1-P15
package.json             root scripts: npm run client | npm run log | npm run lint:html
One app page, e.g. client/pages/dashboard.html, and client/css/main.css + client/css/tokens.css

== CURRENT LAYOUT ==
client/index.html          landing page
client/pages/*.html        24 pages: login, register, dashboard, analytics, expenses, expense-form,
                           expense-edit, income, income-form, budgets, recurring, categories, groups,
                           group-create, group-detail, shared-expense-form, settlements, shopping,
                           shopping-form, shopping-detail, receipt-scan, receipt-confirm, profile, 404
client/css/                main.css imports cascade layers: tokens, base, layout, components, pages, utilities
client/assets/             favicon.svg, sample receipt image
scripts/serve-client.js    dependency-free static server: npm run client -> http://localhost:5500
docs/tools/                build-prompt-log.js (Word log), build_prompt_pdf.py (PDF log)
app.py, config.py, modules/, templates/, static/, database/, ml/, tests/*.py, requirements.txt,
Procfile                   legacy Flask app (reference only)

The Review 1 pages hold realistic hard-coded sample data (user "Priya Raman", October 2026, groups such
as "Goa Trip"). They have no JavaScript yet and the <body> tags have no data-page attribute yet.
Review 1 uses CSS-only tricks that JS will take over: a checkbox + :has() mobile drawer, a
form:has(#split-custom:checked) split toggle, conic-gradient/flexbox charts.

== DECISIONS ALREADY MADE (do not reopen them) ==
- Front end: vanilla JavaScript ES modules. No jQuery, React, Vue, Tailwind, TypeScript or bundler.
- Review 2: client/js/data/api.js is backed by localStorage and has the SAME async function shapes the
  REST API will have, so Review 3 only swaps the data layer to fetch().
- Back end: Node.js 20+, Express, Mongoose. Database: MongoDB Atlas free tier, connection string in
  server/.env (never committed).
- Keep every feature, including receipt OCR (tesseract.js) and the spending forecast.
- Money is stored as integer paise everywhere (client and server).
```

```
== RULES FOR EVERY STEP ==
1.  Do only the step I give you, then stop and report. Do not start the next step.
2.  Before editing, read the files you will touch. Match the existing naming, formatting and comment style.
3.  Keep all links and asset paths relative (../css/main.css, never /css/main.css). The site is served
    from /SplitStack/ on GitHub Pages.
4.  Never put user-supplied data into innerHTML. Use textContent, <template> and createElement.
5.  Format money with Intl.NumberFormat('en-IN', { style: 'currency', currency: 'INR' }).
6.  Reuse the CSS design system. Put new styles in the right layer file, use the tokens in tokens.css,
    no inline style attributes (setting a CSS custom property such as --h from JS is fine). Both the dark
    and the light theme must keep working.
7.  Keep accessibility: labels, aria-* attributes, visible focus, full keyboard use, WCAG AA contrast,
    prefers-reduced-motion.
8.  Pin every CDN script to an exact version (e.g. cdnjs .../Chart.js/4.4.1/chart.umd.min.js).
9.  Do not break Review 1: `npm run lint:html` must still report zero errors.
10. ES modules do not load from file://. From Review 2 on, view the site with `npm run client`
    (http://localhost:5500) or the live site, not by double-clicking the HTML file.
11. Ask me before deleting files, adding a dependency the step does not name, or changing a decision.
12. Report honestly. If a check fails or you skip something, say so with the output.

== LOGGING (mandatory: the teacher reads this log) ==
After each step:
a) Append an object to "entries" in docs/prompt_log.json:
   { "id": <last id + 1>, "date": "YYYY-MM-DD", "stage": "Review 2" | "Review 3" | "Wrap-up",
     "title": "<the step title>", "type": "Working prompt",
     "prompt": "<the prompt I gave you, verbatim>", "purpose": "<one sentence>",
     "files": ["<files created or changed>"], "result": "<what you actually did, incl. problems>",
     "verification": "<commands you ran and their outcome>" }
   If I typed a short request of my own, first add it as a separate entry with "type": "User prompt".
b) Remove the matching item from "planned" (match on its title).
c) Rebuild the logs: `npm run log` (Windows). On macOS/Linux run
   `node docs/tools/build-prompt-log.js && python3 docs/tools/build_prompt_pdf.py`.
   Needs `pip install reportlab`.

== COMMIT, PUSH, REDEPLOY (after each step) ==
git add -A && git commit -m "<message given in the step>"
git push <remote> refurbish refurbish:main          (<remote> = the one pointing at MahaprabhuK/SplitStack)
If anything under client/ changed, republish the static site (fast-forward, no force needed):
git push <remote> $(git subtree split --prefix client refurbish):refs/heads/gh-pages
Then wait about a minute and confirm https://mahaprabhuk.github.io/SplitStack/ still loads.

== YOUR REPORT AT THE END OF EACH STEP ==
What you changed (files), how I can see it working (URL + what to click), the checks you ran with their
results, the commit hash, and anything you could not finish.
```

4. Step prompts P1-P15

Run in order. Each prompt already tells the agent to log, commit, push and stop.

Review 2

P1 JavaScript architecture and app shell

Commit message: `Review 2: JS module architecture and app shell`

```
Step P1 (Review 2): JavaScript architecture and app shell.

Add vanilla JavaScript to the Review 1 pages as native ES modules.
1. Add data-page="<page-name>" to the <body> of index.html and every file in client/pages/
   (e.g. data-page="dashboard", data-page="expense-form"). Load one entry script at the end of each page:
   <script type="module" src="../js/app.js"></script>  (index.html: src="js/app.js").
2. Create client/js/ with:
   - app.js: entry point. Reads document.body.dataset.page, initialises the shared UI, then dynamically
     imports ./pages/<page>.js if it exists and calls its exported init(). A missing page module is not an
     error.
   - core/dom.js: qs, qsa, on, delegate, createEl helpers.
   - core/format.js: formatINR(paise) using Intl.NumberFormat('en-IN', {style:'currency', currency:'INR'}),
     formatDate / formatRelativeDate helpers.
   - ui/theme.js: dark / light / system toggle saved in localStorage, applied to <html data-theme>. Add a
     tiny inline script in each page's <head> that applies the saved theme before first paint (no flash).
     Wire it to a theme button in the topbar (add the button if missing, matching existing button styles).
   - ui/nav.js: replace the CSS checkbox drawer with a real <button aria-expanded aria-controls="sidebar">.
     Close on Escape, scrim click and nav-link click; trap focus inside while open; return focus to the
     button when closed. Keep the page usable without JS.
   - ui/toast.js: accessible toasts (role="status", aria-live="polite"), auto-dismiss, dismiss button.
   - ui/confirm.js: a <dialog>-based confirm that replaces window.confirm for every delete button/form.
3. Add client/js/package.json containing {"type": "module"} so Node (used for unit tests later) treats
   these files as ES modules.
4. All wrapped in try/catch where localStorage is touched (private windows can throw).

Checks: npm run lint:html passes; npm run client, then on dashboard, expenses and groups: theme toggles and
survives reload with no flash; drawer opens/closes with mouse and keyboard at 375px; a delete button opens
the dialog; no console errors on any page.

When you are done: run the checks listed above, then follow LOGGING and COMMIT, PUSH, REDEPLOY from the
project-context prompt. Commit message: "Review 2: JS module architecture and app shell"
Report back and stop.
```

P2 Client data layer backed by localStorage

Commit message: `Review 2: localStorage data layer with seed data`

```
Step P2 (Review 2): client data layer backed by localStorage.

Create client/js/data/ with:
- store.js: keeps collections (users, categories, expenses, income, budgets, recurring, groups,
  sharedExpenses, settlements, shoppingLists, receipts, insights) in localStorage under one versioned key
  (e.g. "splitstack:v1"), generates ids (crypto.randomUUID), handles a missing/corrupt store by reseeding,
  and has a reset() used by a "Reset demo data" button on the profile page.
- seed.js: the SAME sample data the Review 1 pages show. Read the HTML pages (dashboard, expenses,
  income, budgets, recurring, categories, groups, group-detail, settlements, shopping, shopping-detail,
  profile) and transcribe their data: demo user Priya Raman, October 2026, the groups (incl. Goa Trip)
  and their members, the shopping lists and items. Add a few earlier months so analytics has history.
  Amounts in integer paise. Demo login: demo@splitstack.app / Demo@1234 (show this on login.html).
- api.js: async functions with the shape the future REST API will have, grouped as
  auth (register, login, logout, currentUser, updateProfile, changePassword), expenses and income
  (list with filter/sort/pagination, get, create, update, remove), categories, budgets, recurring
  (incl. toggle), groups (list, get, create, addMember, removeMember, leave, sharedExpenses, balances,
  settlements, recordSettlement), shopping (lists, items, toggleItem, convertToExpense), analytics
  (monthlySummary, byCategory, forecast, insights) and search(query).
```

```
  Every function returns a Promise, validates its input and throws an ApiError {field, message, status}
  for bad input, missing records or records owned by someone else. Passwords: hash with
  crypto.subtle SHA-256 + salt (demo only; note in a comment that the server uses bcrypt).
  Page code must import ONLY from api.js, never from store.js.
- Write a short JSDoc block above each function describing arguments and return value. These docs become
  the contract the Review 3 REST API must match.

Checks: in the browser console on localhost:5500, import('/js/data/api.js') and exercise login, create
an expense, list with a filter, and reload to show persistence. Put a short "Data layer" section in
client/README.md listing the api.js functions.

When you are done: run the checks listed above, then follow LOGGING and COMMIT, PUSH, REDEPLOY from the
project-context prompt. Commit message: "Review 2: localStorage data layer with seed data"
Report back and stop.
```

P3 Render pages from data

Commit message: `Review 2: render pages from the data layer`

```
Step P3 (Review 2): render pages from data.

Create one module per page in client/js/pages/ (dashboard.js, expenses.js, income.js, budgets.js,
recurring.js, categories.js, groups.js, group-detail.js, settlements.js, shopping.js, shopping-detail.js,
profile.js, analytics.js ...) and replace the hard-coded sample rows with markup rendered from api.js.
- Use <template> elements placed in the HTML and fill them with textContent. Never innerHTML with data.
- Keep the existing static markup in the HTML as the no-JavaScript fallback: JS clears and re-renders
  the container only after data loads.
- Detail pages read their id from the query string (group-detail.html?id=..., shopping-detail.html?id=...,
  expense-edit.html?id=...). Update every link that points to these pages to include the id.
- Expenses page: filter bar (search text, category, payment method, date range, sort) with debounced
  (250 ms) live filtering, state synced to the query string so reload/back keep the filters,
  pagination (10 per page), and empty states ("No expenses match these filters" + clear-filters button).
- Show the signed-in user's name/avatar initials in the topbar and sidebar.
- Loading state: aria-busy on the container while rendering.

Checks: add, edit and delete via the console API and confirm the pages reflect it; filters work and
survive reload; every page renders with no console errors in both themes at 1366px and 375px.

When you are done: run the checks listed above, then follow LOGGING and COMMIT, PUSH, REDEPLOY from the
project-context prompt. Commit message: "Review 2: render pages from the data layer"
Report back and stop.
```

P4 Forms, validation and CRUD flows

Commit message: `Review 2: forms, validation and CRUD flows`

```
Step P4 (Review 2): forms, validation and CRUD flows.

Wire every form to api.js: register, login, expense add and edit, income, budget, recurring, category,
group create, add member, shared expense, shopping list and item, profile, change password.
- Create client/js/ui/forms.js with reusable helpers: serialise a form to an object (amounts converted
  to paise), show/clear inline errors, map an ApiError {field, message} to the right field.
- Use the Constraint Validation API plus custom rules: passwords match, password strength (8+ chars,
  a letter and a digit), date not in the future for expenses/income, budget > 0, amount > 0 with at most
  2 decimals. Show inline messages linked with aria-describedby and aria-invalid, and move focus to the
  first invalid field on submit.
- After a successful save: toast, then redirect to the list page (Post/Redirect/Get). Edit pages
  pre-fill from the record. Delete uses the confirm dialog from P1.
- Password show/hide toggles: a <button aria-pressed> next to each password field.
- Auth guard: every app page except index, login, register and 404 redirects to login.html?next=<page>
  when nobody is signed in; login returns to "next". Logout clears the session.
- Disable the submit button while saving to prevent double submits.

Checks: walk through each form with invalid then valid input using only the keyboard; check the error
messages are announced (aria-describedby wired); confirm redirects and toasts; confirm the auth guard.

When you are done: run the checks listed above, then follow LOGGING and COMMIT, PUSH, REDEPLOY from the
```

```
project-context prompt. Commit message: "Review 2: forms, validation and CRUD flows"
Report back and stop.
```

P5 Group splitting and the debt-simplification algorithm

Commit message: `Review 2: split logic and debt simplification with tests`

```
Step P5 (Review 2): group splitting and the debt-simplification algorithm.

1. client/js/domain/splits.js
   - equalSplit(totalPaise, memberIds, payerId): integer paise per member; the leftover paise go to the
     payer so the shares sum exactly to the total.
   - validateCustomSplit(totalPaise, shares): shares must be non-negative integers summing exactly to the
     total; return {ok, remaining}.
2. On shared-expense-form.html: switch equal/custom split with JS (keep the CSS :has() version as the
   no-JS fallback), show a live "Remaining to assign: Rs. x" counter (aria-live), and disable submit until
   remaining is exactly zero. Equal mode shows each member's computed share.
3. client/js/domain/settle.js
   - netBalances(sharedExpenses, settlements): per member, paid minus share, adjusted by settlements.
   - simplifyDebts(balances): greedy minimum-cash-flow. Repeatedly match the largest debtor with the
     largest creditor, transfer min(debt, credit), until every balance is within 1 paisa. Return
     {transfers: [{from, to, amount}], originalDebtCount} where originalDebtCount is the number of
     pairwise debts before simplification.
   - Both are pure functions with no DOM or storage access, because the server will import them in P11.
4. Render group-detail balances and settlements.html from these functions: show "N payments instead of
   M". "Mark paid" records a settlement via api.js and recomputes everything.
5. Unit tests in tests/client/splits.test.mjs and tests/client/settle.test.mjs using node:test and
   node:assert. Include the Goa Trip example: balances +1700, -2500, +5700, -4900 (rupees) must produce
   exactly 3 transfers that settle everyone; also equal split with remainder, an invalid custom split,
   all-zero balances, and a single debtor/creditor pair. Add the script "test:client":
   "node --test tests/client/" to the root package.json.

Checks: npm run test:client passes (paste the output); settle-up on the Goa Trip group matches the tests.

When you are done: run the checks listed above, then follow LOGGING and COMMIT, PUSH, REDEPLOY from the
project-context prompt. Commit message: "Review 2: split logic and debt simplification with tests"
Report back and stop.
```

P6 Charts, budgets, forecast, insights, search and notifications

Commit message: `Review 2: charts, budgets, forecast, insights, search, notifications`

```
Step P6 (Review 2): charts, budgets, forecast, insights, search and notifications.

- Charts: load Chart.js 4 from a pinned cdnjs URL only on dashboard and analytics. Build a doughnut
  (spend by category), a bar chart (monthly income vs expenses, last 6 months) and a line chart (daily
  spend this month vs forecast). Read colours from CSS custom properties with getComputedStyle so both
  themes work, and re-render on theme change. Keep the CSS charts as the fallback when Chart.js fails to
  load. Give each canvas an aria-label and a visually hidden data table.
- Budgets: status thresholds ok < 75%, caution 75-90%, warning 90-100%, danger > 100%. Update the meters
  (value + class), and show "Rs. x left" or "Rs. x over".
- client/js/domain/forecast.js: projected month spend = average daily spend so far x days in month.
  Port any extra rules from the legacy ml/ and modules/ Python code if they exist (read them first).
  Add tests in tests/client/forecast.test.mjs.
- Insights (client/js/domain/insights.js): over-budget categories, budgets near their limit,
  month-over-month change per category (>= 20%), recurring bills due in the next 7 days, unsettled group
  balances. Show them on the dashboard.
- Global search (client/js/ui/search.js): the topbar search shows live results across expenses, income,
  groups and shopping lists in a listbox (role="combobox"/"listbox", aria-activedescendant), with
  ArrowUp/ArrowDown/Enter/Escape support; debounce 200 ms; Enter on a result opens it.
- Notifications (client/js/ui/notifications.js): the bell opens a panel built from the insights, with an
  unread count badge; "Mark all read" persists via api.js.

Checks: npm run test:client; charts render in both themes and when you block the CDN (fallback shown);
search and notifications fully usable by keyboard.

When you are done: run the checks listed above, then follow LOGGING and COMMIT, PUSH, REDEPLOY from the
```

```
project-context prompt. Commit message: "Review 2: charts, budgets, forecast, insights, search, notifications"
Report back and stop.
```

P7 Shopping checklist and receipt scan preview

Commit message: `Review 2: shopping checklist and receipt OCR preview`

```
Step P7 (Review 2): shopping checklist and receipt scan preview.

Shopping (client/js/pages/shopping-detail.js):
- Toggle items purchased with optimistic UI (update the DOM first, roll back with a toast if api.js
  throws). Edit actual price inline (Enter saves, Escape cancels). Recompute estimated, actual and
  variance totals live, and the list's progress meter.
- "Convert to expense": creates one expense for the actual total in the chosen category, marks the list
  converted, and must not create a duplicate if clicked twice or after reload (button becomes a link to
  the created expense).

Receipt scan (client/js/pages/receipt-scan.js + client/js/domain/receipt-parser.js):
- Drag-and-drop zone with dragenter/dragover/dragleave/drop styling, plus the normal file input and a
  keyboard-accessible "Choose file" button.
- Client-side checks: PNG, JPEG or WEBP only, max 5 MB, clear error messages.
- Preview with URL.createObjectURL (revoke it afterwards).
- OCR in the browser with tesseract.js from a pinned jsDelivr URL, with a progress bar.
- receipt-parser.js (pure function, will be reused on the server): from the OCR text extract merchant
  (first meaningful line), date (common Indian formats dd/mm/yyyy, dd-mm-yy, 12 Oct 2026) and total
  (prefer the line containing TOTAL / GRAND TOTAL / NET AMOUNT, otherwise the largest amount). Tests in
  tests/client/receipt-parser.test.mjs with 4+ sample texts.
- Store the result and open receipt-confirm.html pre-filled; the user can correct fields; confirming
  creates the expense. Nothing becomes an expense without confirmation.
- Test with client/assets' sample receipt image.

Checks: npm run test:client; convert-to-expense cannot duplicate; OCR on the sample receipt fills the
confirm form.

When you are done: run the checks listed above, then follow LOGGING and COMMIT, PUSH, REDEPLOY from the
project-context prompt. Commit message: "Review 2: shopping checklist and receipt OCR preview"
Report back and stop.
```

P8 Review 2 QA, log and commit

Commit message: `Review 2: QA, lint, docs`

```
Step P8 (Review 2): QA, documentation and close Review 2.

1. ESLint: add eslint, @eslint/js and globals as root devDependencies and an eslint.config.js (flat config,
   recommended rules, browser globals for client/js, node globals for tests, ecmaVersion 2023, sourceType
   module). Add "lint:js": "eslint client/js tests/client". Fix every finding.
2. Run npm run lint:html, npm run lint:js and npm run test:client. All must pass.
3. Manual pass on EVERY page at 1366px and 375px, in dark and light theme, keyboard only, and with
   JavaScript disabled (pages must still show their Review 1 static content). Fix what you find and list
   what you checked in the log entry.
4. client/README.md: add "Review 2 - JavaScript" with the module structure (tree), how data flows
   (page -> api.js -> store.js -> localStorage), the demo login, how to run the tests, and a table
   "JavaScript concept -> where it is used" (ES modules, DOM manipulation, events and delegation,
   <template>, Constraint Validation API, async/await and Promises, localStorage/JSON, dynamic import,
   Intl, <dialog>, drag-and-drop and File API, debouncing, algorithms, unit tests, Chart.js).
5. docs/REVIEW2_DEMO.md: a 5-minute demo script for the review (what to click, what to say, which code
   to show).
6. Update the Review status line at the top of the root README.md.
7. Log, commit, push, and redeploy gh-pages. Open the live site and confirm a full flow works there
   (login with the demo user, add an expense, settle up).

When you are done: run the checks listed above, then follow LOGGING and COMMIT, PUSH, REDEPLOY from the
project-context prompt. Commit message: "Review 2: QA, lint, docs"
Report back and stop.
```

Checkpoint: Review 2 is complete here. Rehearse with the Review rehearsal prompt before the review.

Review 3

P9 Express server and MongoDB connection

Commit message: `Review 3: Express server and MongoDB connection`

```
Step P9 (Review 3): Express server and MongoDB connection.

Create server/ as its own Node.js 20+ project with "type": "module":
- server/package.json with scripts: "dev": "node --watch src/index.js", "start": "node src/index.js".
  Dependencies: express, mongoose, express-session, connect-mongo, helmet, compression, express-rate-limit,
  dotenv. Add others only if a later step names them.
- server/src/config.js: read and validate env vars (MONGODB_URI required, SESSION_SECRET at least 32
  chars, PORT default 3000, NODE_ENV). Fail fast with a clear message if missing.
- server/src/app.js (exports the app, no listen, so tests can import it):
  helmet with a Content Security Policy that allows only self plus the exact CDNs the client uses
  (Google Fonts, cdnjs for Font Awesome and Chart.js, jsDelivr for tesseract.js) - check the pages to get
  the full list; compression; express.json({limit: '100kb'}); express-session stored in MongoDB via
  connect-mongo (httpOnly, sameSite 'lax', secure in production, 7-day maxAge); express-rate-limit on
  /api/auth and upload routes; CSRF protection (csurf is deprecated: implement a small synchronizer
  token: GET /api/csrf returns a token stored in the session; every POST/PUT/PATCH/DELETE must send it in
  an X-CSRF-Token header); GET /api/health -> {ok: true}; express.static serving ../client; a 404 handler
  for /api; a central error handler that returns JSON {error, fields} and never leaks stack traces in
  production.
- server/src/db.js: connect with Mongoose, log the database name (not the URI), close cleanly on SIGINT.
- server/src/index.js: load config, connect, listen.
- server/.env.example with MONGODB_URI (use a database name ending in -dev, e.g.
  .../splitstack-dev), SESSION_SECRET, PORT. Confirm .env is git-ignored. Never commit secrets.
- server/README.md: setup steps including creating a free MongoDB Atlas cluster, a database user, and
  network access.

I will put my Atlas connection string in server/.env myself; ask me for it if it is not there. Do not
print it.

Checks: cd server && npm install && npm run dev; GET /api/health returns ok; the client pages load from
http://localhost:3000 with no CSP errors in the console.

When you are done: run the checks listed above, then follow LOGGING and COMMIT, PUSH, REDEPLOY from the
project-context prompt. Commit message: "Review 3: Express server and MongoDB connection"
Report back and stop.
```

P10 Mongoose data models

Commit message: `Review 3: Mongoose data models`

```
Step P10 (Review 3): Mongoose data models.

Read the legacy SQLite schema (database/ and modules/) and the Review 2 api.js contract, then create
server/src/models/ with one file per model, schema validation, timestamps and indexes:
- User: name, email (unique, lowercase, trimmed), passwordHash, preferences {theme, currency}.
- Category: user (null for built-in defaults), name, type 'expense'|'income', icon, color.
  Unique on user+name+type.
- Expense / Income: user, category (ref), amount (integer paise, min 1), date, description / source,
  paymentMethod (enum). Index user+date.
- Budget: user, category (ref or null for overall), month, year, limit (paise). Unique on
  user+category+month+year.
- Recurring: user, name, amount, category, frequency (daily|weekly|monthly|yearly), startDate,
  nextDueDate, active.
- Group: name, description, createdBy, members [{user, role 'admin'|'member', joinedAt}] (embedded).
- SharedExpense: group, title, amount, paidBy, category, date, splitType 'equal'|'custom',
  shares [{user, amount}] (embedded) with a validator that shares sum exactly to amount.
- Settlement: group, payer, payee, amount, status 'pending'|'completed', completedAt.
- ShoppingList: owner, optional group, name, budget, items [{name, quantity, estimatedPrice,
  actualPrice, purchased, assignedTo}] (embedded), convertedExpense (ref, set once).
- Receipt: user, fileId (GridFS), merchant, date, amount, rawText, confidence, status.
- Insight: user, kind, message, read.
```

```
At the top of each file, a short comment explaining what is embedded vs referenced and why. Add
toJSON transforms that rename _id to id and drop passwordHash and __v so responses match api.js.

Also add docs/DATA_MODEL.md with a Mermaid ER diagram of the collections.

Checks: a small script (server/scripts/check-models.js) that connects, calls Model.syncIndexes() for
all models, and creates and deletes one document of each against the -dev database. Paste its output.

When you are done: run the checks listed above, then follow LOGGING and COMMIT, PUSH, REDEPLOY from the
project-context prompt. Commit message: "Review 3: Mongoose data models"
Report back and stop.
```

P11 Authentication and REST API

Commit message: `Review 3: authentication and REST API`

```
Step P11 (Review 3): authentication and REST API.

Implement the REST API under /api, one express.Router per module in server/src/routes/, business logic
in server/src/services/, shared middleware in server/src/middleware/ (requireAuth, validate, loadGroup /
requireMember / requireAdmin, asyncHandler). Endpoints must match the Review 2 api.js function list and
cover the 48 routes of the legacy Flask app (read app.py and modules/ to check nothing is missing):
- /api/auth: register (bcrypt cost 12), login (regenerate the session id), logout, me, update profile,
  change password.
- /api/expenses, /api/income: list with filter/sort/pagination query params, get, create, update, delete.
- /api/categories, /api/budgets (with computed status), /api/recurring (incl. PATCH toggle).
- /api/groups: CRUD, members (add by email, remove, leave), shared expenses, balances, settlements
  (record, mark paid).
- /api/shopping: lists, items, toggle, convert to expense (only once).
- /api/analytics: monthly, by category, forecast, insights.   /api/search?q=
Requirements:
- Validate every body/query/params with zod (add the dependency). Return 400 with {error, fields}.
- Ownership on every request: users only see their own data; group data only for members; admin-only
  actions checked. Return 404 (not 403) for records that belong to someone else.
- Use MongoDB transactions (session.withTransaction) for multi-document writes: recording a settlement,
  converting a shopping list. (Atlas supports transactions; the in-memory test server must run as a
  replica set in P14.)
- Reuse the Review 2 pure modules on the server: import client/js/domain/splits.js, settle.js,
  forecast.js and insights.js directly so each algorithm has one implementation.
- docs/API.md: table of every endpoint (method, path, auth, body, response).

Checks: a REST client file (server/requests.http) or curl script exercising register, login (with CSRF
token), CRUD on expenses, group create + shared expense + balances + settlement; paste the results.

When you are done: run the checks listed above, then follow LOGGING and COMMIT, PUSH, REDEPLOY from the
project-context prompt. Commit message: "Review 3: authentication and REST API"
Report back and stop.
```

P12 Receipt upload and server-side OCR

Commit message: `Review 3: receipt upload and server-side OCR`

```
Step P12 (Review 3): receipt upload and server-side OCR.

Add multer, sharp and tesseract.js to server/.
- POST /api/receipts: multer memory storage, 5 MB limit, one file; verify magic bytes (PNG, JPEG, WEBP),
  not just the extension or mimetype.
- server/src/services/ocr.js: preprocess with sharp (rotate from EXIF, greyscale, upscale images narrower
  than 1000px, normalise), run tesseract.js with ONE shared worker created at startup and terminated on
  shutdown, then parse with the Review 2 client/js/domain/receipt-parser.js.
- Store the image in MongoDB GridFS (mongoose.mongo.GridFSBucket) so it survives redeploys on free
  hosting, and the parsed fields in a Receipt document with status 'pending'.
- GET /api/receipts/:id and GET /api/receipts/:id/image (streams the image, owner only).
- POST /api/receipts/:id/confirm with the user-corrected fields creates the expense and marks the receipt
  confirmed. Nothing becomes an expense without this call; confirming twice must not duplicate.
- Rate-limit the upload route.
```

```
Checks: upload the sample receipt from client/assets with curl; reject a renamed .txt file and a >5 MB
file; confirm creates exactly one expense.

When you are done: run the checks listed above, then follow LOGGING and COMMIT, PUSH, REDEPLOY from the
project-context prompt. Commit message: "Review 3: receipt upload and server-side OCR"
Report back and stop.
```

P13 Connect the client to the API

Commit message: `Review 3: client uses the REST API`

```
Step P13 (Review 3): connect the client to the API.

The live GitHub Pages site has no server, so it must keep working as a localStorage demo while the
Node server uses the real API.
- Rename the Review 2 localStorage implementation to client/js/data/api-local.js (unchanged).
- Write client/js/data/api-http.js with the SAME exports, using fetch() against /api: credentials
  'same-origin', JSON bodies, fetch and cache the CSRF token from GET /api/csrf and send it as
  X-CSRF-Token, convert amounts consistently (paise), map HTTP errors to the same ApiError
  {field, message, status} the pages already handle, redirect to login on 401, and show a toast when
  offline (navigator.onLine / failed fetch).
- client/js/data/api.js becomes a small switch: on startup call GET /api/health with a short timeout;
  if it answers, re-export api-http.js, otherwise api-local.js. Show a small "Demo mode - data is saved
  in this browser" badge in demo mode.
- Page modules must NOT need changes. If one does, explain why in the log.
- Remove the client-side OCR path when the server is available (receipt-scan posts the file to
  /api/receipts); keep it for demo mode.

Checks: run the server against Atlas and walk every flow end to end (register, login, expenses, income,
budgets, recurring, categories, groups + settle-up, shopping + convert, receipt scan, analytics, search,
profile, logout). Then confirm the GitHub Pages site still works in demo mode after redeploying gh-pages.

When you are done: run the checks listed above, then follow LOGGING and COMMIT, PUSH, REDEPLOY from the
project-context prompt. Commit message: "Review 3: client uses the REST API"
Report back and stop.
```

P14 Seed data and automated tests

Commit message: `Review 3: seed script, API tests and CI`

```
Step P14 (Review 3): seed data and automated tests.

1. server/scripts/seed.js (npm run seed): creates the demo user (demo@splitstack.app / Demo@1234),
   default categories, six months of expenses and income, budgets, recurring payments, the three groups
   with members and shared expenses, and shopping lists - consistent with the Review 2 seed data.
   It must be idempotent (running twice gives the same data) and must REFUSE to run when the database
   name does not end in -dev or -test, unless --force is passed.
2. Tests with Jest + Supertest + mongodb-memory-server (MongoMemoryReplSet, because the API uses
   transactions) in server/tests/: auth (register, login, logout, wrong password, duplicate email),
   ownership (user B cannot read/edit user A's expense; non-member cannot see a group), CSRF rejection,
   rate limiting, validation errors, split validation, settlement algorithm through the API, budget
   status, convert-to-expense duplicate prevention, receipt upload validation (type, size, magic bytes).
   Cover at least what the 47 legacy Python tests in tests/ checked (read them). Add "test" and
   "test:coverage" scripts. Jest with ES modules needs NODE_OPTIONS=--experimental-vm-modules; use
   cross-env so it works on Windows.
3. .github/workflows/ci.yml: on push and pull_request, Node 20, npm ci at the root and in server/,
   run lint:html, lint:js, test:client and the server tests.
   Note: pushing a workflow file needs a GitHub token with the "workflow" scope. If the push is
   rejected, tell me; I will run `gh auth refresh -s workflow` and you push again.
4. docs/REVIEW3_DEMO.md: a 5-minute demo script for Review 3 (Atlas collections, API calls, tests).

Checks: npm test in server/ passes (paste the summary and coverage); seed runs twice without changing
counts; seed refuses a database name without -dev/-test.

When you are done: run the checks listed above, then follow LOGGING and COMMIT, PUSH, REDEPLOY from the
project-context prompt. Commit message: "Review 3: seed script, API tests and CI"
Report back and stop.
```

Checkpoint: Review 3 is complete here. Rehearse with the Review rehearsal prompt before the review.

Wrap-up

P15 Remove legacy code, document and deploy

Commit message: `Wrap-up: remove legacy Flask app, docs and deployment`

```
Step P15 (Wrap-up): remove legacy code, document and deploy.

1. First list the legacy Flask files you plan to delete (app.py, config.py, database/, modules/,
   templates/, static/, ml/, tests/*.py, requirements.txt, Procfile, uploads/ if unused) and confirm that
   the `legacy` branch on GitHub still has them. Wait for my OK, then delete them from refurbish.
2. Rewrite the root README.md: what SplitStack does, screenshots (save to docs/screenshots/), the stack
   per review (HTML+CSS / JavaScript / Node.js + MongoDB), how to run (client-only demo and full
   stack), environment variables, the data model (link docs/DATA_MODEL.md), the API (link docs/API.md),
   tests, live links, and the team.
3. Deployment on Render: add render.yaml (web service, rootDir server, build "npm ci", start "npm start",
   env vars MONGODB_URI and SESSION_SECRET marked sync:false, NODE_ENV=production, health check
   /api/health). Make sure the app trusts the proxy (app.set('trust proxy', 1)) so secure cookies work.
   Write the exact manual steps for me (Render dashboard, Atlas network access 0.0.0.0/0, setting env
   vars) in docs/DEPLOY.md - I will do the account steps myself. After I give you the Render URL, check
   it end to end.
4. Keep the GitHub Pages demo: redeploy gh-pages and check it.
5. Finish the prompt log (all planned items should now be gone), rebuild Word + PDF.
6. Commit, push refurbish and main, then tag: git tag -a v3.0.0 -m "SplitStack 3.0.0" and push the tag.

When you are done: run the checks listed above, then follow LOGGING and COMMIT, PUSH, REDEPLOY from the
project-context prompt. Commit message: "Wrap-up: remove legacy Flask app, docs and deployment"
Report back and stop.
```

5. Utility prompts

Status check (use any time, e.g. at the start of a new session)

```
Read docs/prompt_log.json, `git status`, `git log --oneline -15` and the remote branches. Tell me:
which steps P1-P15 are done, which is next, whether the working tree is clean, whether refurbish, main
and the remote are in sync, and whether gh-pages is behind client/. Do not change anything.
```

Recovery (when the agent went off track or something broke)

```
Stop. Do not make further changes. Run `git status` and `git diff --stat` and explain exactly what you
changed in this step and why, what is broken, and the smallest fix. Do not revert, reset or delete
anything until I approve your plan. Re-read the RULES section of the project-context prompt and tell me
which rule was broken, if any.
```

Review rehearsal (before Review 2 or Review 3)

```
Act as the course examiner for Review <2 or 3>. Using docs/REVIEW<2 or 3>_DEMO.md, run through the demo
yourself on the running app, list anything that fails or looks unfinished, and write 10 likely viva
questions about the code with short, correct answers that point to the exact files. Fix nothing yet;
give me the list first.
```

6. Things that commonly go wrong

- Pages open but nothing happens: ES modules do not run from file://. Use npm run client or the server.
- Live site shows old content: client/ changed but gh-pages was not republished (see Prompt 0, COMMIT, PUSH, REDEPLOY). Pages can take a minute to update; hard-refresh.
- Broken styles or 404s on the live site only: an absolute path (/css/...) slipped in. Paths must be relative.
- Push rejected with "workflow scope": the token cannot push .github/workflows files. Run gh auth refresh -s workflow, then push again.
- Node tests fail with "Cannot use import statement": client/js/package.json with {"type": "module"} is missing.
- MongoDB connection fails: your current IP is not in Atlas Network Access, or the password in the URI contains special characters that need URL encoding.
- Transactions fail in tests: mongodb-memory-server must start as a replica set (MongoMemoryReplSet).
- Money totals off by one paisa: some code used rupee floats. All amounts are integer paise.
- Prompt log out of date: every step must add an entry and run npm run log. The teacher reads this log.